

Assignemt-08 (Lab-8.5)

Course : AI Assisted Coding

Topic:- Test-Driven Development with AI – Generating and Working with Test Cases

Student Details

Name : Batti Chandan Singh
Hall Ticket No : 2303A51515
Batch : 22

Task-01

Final Optimal Prompt

#Generate the python function about checking user enter the UserName is valid or not ,run the assertion test cases ,

#the userName contains both string and numbers, the lenght of String and numbers should be between 5 to 15 characters,

if user enter the wrong input then it will show the error message proper include the what he/she enter indication.

#eg: input: "user123" output: "Valid UserName - True"

#eg: input: "12user" ouput:"invalid userName - False"

#eg: input: "user_123" output: "invalid userName - False"

#eg: input: "user 12345" output: "invalid userName - False"

#eg: input: "us" output: "Error: UserName must be between 5 and 15 characters long with numbers and letters only."

#input taking from user

Code Screenshot

```

11
12 def is_valid_username(username):
13     if len(username) < 5 or len(username) > 15:
14         return f"Error: UserName must be between 5 and 15 characters long with numbers and letters only. You entered: '{username}'"
15     if not username[0].isalpha():
16         return f"invalid userName - False. You entered: '{username}'"
17     if not username.isalnum():
18         return f"invalid userName - False. You entered: '{username}'"
19     has_letter = any(char.isalpha() for char in username)
20     has_number = any(char.isdigit() for char in username)
21     if not (has_letter and has_number):
22         return f"invalid userName - False. You entered: '{username}'"
23     return "Valid UserName - True"
24
25 assert is_valid_username("user123") == "Valid UserName - True"
26 assert is_valid_username("12user") == "invalid userName - False. You entered: '12user'"
27 assert is_valid_username("user_123") == "invalid userName - False. You entered: 'user_123'"
28 assert is_valid_username("user 12345") == "invalid userName - False. You entered: 'user 12345'"
29 assert is_valid_username("us") == "Error: UserName must be between 5 and 15 characters long with numbers and letters only. You entered: 'us'"
30
31 #Taking input from user
32 user_input = input("Enter a UserName: ")
33 result = is_valid_username(user_input)
34 print(result)

```

Output Screenshot

```

● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "python file/Assignment-08(Lab-8.5).py"
d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-08(Lab-8.5).py
pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
Enter a UserName: chandu
invalid userName - False. You entered: 'chandu'
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "python file/Assignment-08(Lab-8.5).py"
d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-08(Lab-8.5).py
pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
Enter a UserName: chandu123
Valid UserName - True
○ PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This program validates a username based on rules such as length (5–15 characters), starting with an alphabet, and containing only letters and digits. It also checks that the username includes at least one letter and one number. Assertion tests ensure the function behaves correctly for valid and invalid usernames. Error messages clearly show what the user entered, helping identify mistakes. User input is also handled safely to prevent unexpected issues.

Task-02

Final Optimal Prompt

#Generate the python function name "classify_value(x)" to check whether given number is Odd or Even, run the assertion test cases,

#if user enter the wrong input then it will show the error message proper include the what he/she enter indication.

#eg: input: 5 output: "5 is Odd"

#eg: input: 8 output: "8 is Even"

#eg: input: "abc" output: "Invalid input - Please enter a valid integer(Number). You entered: 'abc'"

#eg: input: 3.14 output: "Invalid input - Please enter a valid integer(Number). You entered: '3.14'"

#eg: input: 0 output: "zero"

Code Screenshot

```
44 #eg: input: 0 output: zero
45 def classify_value(x):
46     if isinstance(x, int):
47         if x == 0:
48             return "zero"
49         elif x % 2 == 0:
50             return f"{x} is Even"
51         else:
52             return f"{x} is odd"
53     else:
54         return f"Invalid input - Please enter a valid integer(Number). You entered: '{x}'"
55 assert classify_value(5) == "5 is Odd"
56 assert classify_value(8) == "8 is Even"
57 assert classify_value("abc") == "Invalid input - Please enter a valid integer(Number). You entered: 'abc'"
58 assert classify_value(3.14) == "Invalid input - Please enter a valid integer(Number). You entered: '3.14'"
59 assert classify_value(0) == "zero"
60
61 #Taking input from user
62 user_input = input("Enter a value: ")
63 try:
64     user_input = int(user_input)
65 except ValueError:
66     pass
67 result = classify_value(user_input)
68 print(result)
```

Output Screenshot

```
ents python file/Assignment-08(Lab-8.5).py"
d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-08(
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
Enter a value: 12
12 is Even
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\pyth
ents python file/Assignment-08(Lab-8.5).py"
d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-08(
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
Enter a value: u
Invalid input - Please enter a valid integer(Number). You entered: 'u'
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\pyth
ents python file/Assignment-08(Lab-8.5).py"
d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-08(
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
Enter a value: 9.7
Invalid input - Please enter a valid integer(Number). You entered: '9.7'
○ PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The `classify_value()` function checks whether a given input is an integer and classifies it as "Even," "Odd," or "Zero." If the user enters text, decimals, or invalid values, a clear error message is returned with the incorrect input shown. Assertion test cases verify correct behavior across multiple scenarios. The program uses exception handling when converting user input to maintain robustness. This ensures reliable classification and meaningful feedback for all types of inputs.

Task-03

Final Optimal Prompt

Generate the python function name "`is_palindrome(text)`" to check whether given string is palindrome or not, run the assertion test cases,

include the proper error message if user enter the wrong input. ignore the space in between sentence, punctuation and case sensitivity.

eg: input: 'madam' output: "True - madam is a palindrome"

eg: input: "A man, a plan, a canal, Panama!" output: "True - A man, a plan, a canal, Panama! is a palindrome"

eg: input: "Hello" output: "False - Hello is not a palindrome"

eg: input: 12321 output: "Invalid input - Please enter a valid string. You entered: '12321'"

eg: input: "No 'x' in Nixon" output: "True - No 'x' in Nixon is a palindrome"

Code Screenshot

```
78 '''
79 def is_palindrome(text):
80     if not isinstance(text, str):
81         return f"Invalid input - Please enter a valid string. You entered: '{text}'"
82     cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
83     if cleaned_text == cleaned_text[::-1]:
84         return f"True - {text} is a palindrome"
85     else:
86         return f"False - {text} is not a palindrome"
87 assert is_palindrome('madam') == "True - madam is a palindrome"
88 assert is_palindrome("A man, a plan, a canal, Panama!") == "True - A man, a plan, a canal, Panama! is a palindrome"
89 assert is_palindrome("Hello") == "False - Hello is not a palindrome"
90 assert is_palindrome(12321) == "Invalid input - Please enter a valid string. You entered: '12321'"
91 assert is_palindrome("No 'x' in Nixon") == "True - No 'x' in Nixon is a palindrome"
92
93 print("All assertion test cases passed successfully.")
94
95 #Taking input from user
96 user_input = input("Enter a string: ")
97 result = is_palindrome(user_input)
98 print(result)
```

Output Screenshot

```

All assertion test cases passed successfully.
Enter a string: madam
True - madam is a palindrome
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\p
ents python file/Assignment-08(Lab-8.5).py"
d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-
pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
All assertion test cases passed successfully.
Enter a string: 123
False - 123 is not a palindrome
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\p
ents python file/Assignment-08(Lab-8.5).py"
d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-
pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
All assertion test cases passed successfully.
Enter a string: une;
False - une; is not a palindrome
○ PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> █

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The palindrome function removes spaces, punctuation, and ignores case to correctly detect palindromes. It handles both simple and complex sentences while ensuring accurate comparison by normalizing the text. If users enter invalid data such as numbers or non-string types, a helpful error message is displayed. Assertion tests verify correct results for valid, invalid, and edge-case inputs. The program ensures reliable string validation and user-friendly error handling.

Task-04

Final Optimal Prompt

Generate assert test cases for a BankAccount class with deposit, withdraw, and get_balance methods. Include negative amount checks, insufficient balance handling, and edge cases

Code Screenshot

```

class BankAccount:
    def deposit(self, amount):
        self.balance += amount
        return f"Deposited: {amount}. Current balance: {self.balance}"
    def withdraw(self, amount):
        if amount <= 0:
            return f"Invalid withdrawal amount. You entered: {amount}"
        if amount > self.balance:
            return f"Insufficient balance. You attempted to withdraw: {amount}, but current balance is: {self.balance}"
        self.balance -= amount
        return f"Withdrew: {amount}. Current balance: {self.balance}"
    def get_balance(self):
        return f"Current balance: {self.balance}"

# Assertion test cases
account = BankAccount()
assert account.deposit(100) == "Deposited: 100. Current balance: 100"
assert account.deposit(-50) == "Invalid deposit amount. You entered: -50"
assert account.withdraw(30) == "Withdrew: 30. Current balance: 70"
assert account.withdraw(100) == "Insufficient balance. You attempted to withdraw: 100, but current balance is: 70"
assert account.withdraw(-20) == "Invalid withdrawal amount. You entered: -20"
assert account.get_balance() == "Current balance: 70"
print("All assertion test cases passed successfully.")

# Taking input from user
account = BankAccount()
while True:
    action = input("Enter action (deposit, withdraw, balance, exit): ").lower()
    if action == "deposit":
        amount = float(input("Enter amount to deposit: "))
        print(account.deposit(amount))
    elif action == "withdraw":
        amount = float(input("Enter amount to withdraw: "))
        print(account.withdraw(amount))
    elif action == "balance":
        print(account.get_balance())
    elif action == "exit":
        print("Exiting the program.")
        break
    else:
        print("Invalid action. Please enter 'deposit', 'withdraw', 'balance', or 'exit'.")

```

Output Screenshot

```

PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\python
ents python file/Assignment-08(Lab-8.5).py"
d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-08(L
pattern = r'^[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
All assertion test cases passed successfully.
Enter action (deposit, withdraw, balance, exit): deposit
Enter amount to deposit: 129292
Deposited: 129292.0. Current balance: 129292.0
Enter action (deposit, withdraw, balance, exit): withdraw
Enter amount to withdraw: 499
Withdrew: 499.0. Current balance: 128793.0
Enter action (deposit, withdraw, balance, exit): balance
Current balance: 128793.0
Enter action (deposit, withdraw, balance, exit): exit
Exiting the program.
PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task uses Test-Driven Development by creating assertion test cases before writing the BankAccount class. The class supports depositing, withdrawing, and checking balance, with

safeguards for negative amounts and insufficient funds. Each method returns meaningful messages to guide the user. Assertion tests confirm correct behavior across normal and edge cases. The interactive part allows users to perform banking actions safely with proper input handling.

Tack-05

Final Optimal Prompt

Generate assert test cases for `validate_email(email)`. Requirements: must include @ and cannot start/end with special characters, handle invalid formats safely.inclde proper error message if user enter the wrong input.

Email validation with error handling + test cases

Code Screenshot

```
def validate_email(email):
    # Type check
    if not isinstance(email, str):
        return f"Invalid input - Please enter a valid email string. You entered: '{email}'"
    email = email.strip()
    # Empty input
    if email == "":
        return "Error: Email cannot be empty."
    # Cannot start or end with special characters
    if email[0] in "@._-" or email[-1] in "@._-":
        return f"Invalid email address - Cannot start or end with special characters. You entered: '{email}'"

    # Basic email regex
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$'
    if bool(re.match(pattern, email)):
        return f"Valid email address: {email}"
    else:
        return f"Invalid email address: {email}"

# ----- Assertion test cases -----
assert validate_email("user@example.com") == "Valid email address: user@example.com"
assert validate_email("userexample.com") == "Invalid email address: userexample.com"
assert validate_email("user@.com") == "Invalid email address: user@.com"
assert validate_email("@example.com") == "Invalid email address - Cannot start or end with special characters. You entered: '@example.com'"
assert validate_email("user@com") == "Invalid email address: user@com"
assert validate_email(".user@gmail.com") == "Invalid email address - Cannot start or end with special characters. You entered: '.user@gmail.com'"
assert validate_email("user@com.") == "Invalid email address - Cannot start or end with special characters. You entered: 'user@com.'"

print("All assertion test cases passed successfully.")
# ----- User Input + Error Handling -----
try:
    user_input = input("Enter an email address: ").strip()
    result = validate_email(user_input)
    print(result)
except KeyboardInterrupt:
    print("\nProgram interrupted by user.")
except Exception as e:
    print(f"Unexpected error occurred: {e}")
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> &
ents python file/Assignment-08(Lab-8.5).py"
Valid email address: chandy@113gmail.com
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> &
ents python file/Assignment-08(Lab-8.5).py"
All assertion test cases passed successfully.
Enter an email address: company@.edu.in
Valid email address: company@.edu.in
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> &
ents python file/Assignment-08(Lab-8.5).py"
All assertion test cases passed successfully.
Enter an email address: 12@.in
Invalid email address: 12@.in
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> &
ents python file/Assignment-08(Lab-8.5).py"
All assertion test cases passed successfully.
Enter an email address: chie.in
Invalid email address: chie.in
○ PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> |
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The email validator checks format rules such as presence of '@', proper domain, and restrictions on starting or ending with special characters. It safely handles invalid inputs like empty strings, wrong formats, or non-string values. Assertion test cases verify accurate validation for both valid and invalid emails. User input includes clear error messages when mistakes occur. This ensures reliable, secure, and user-friendly email validation behavior.